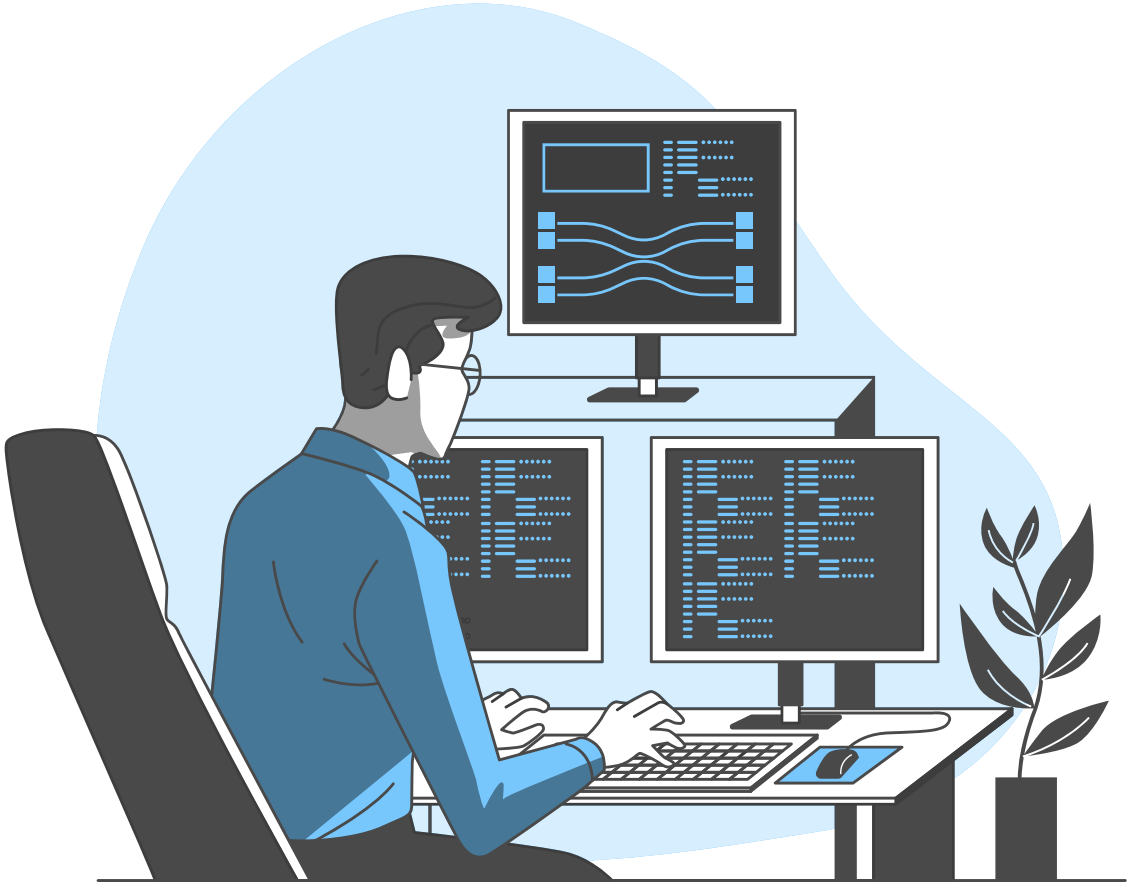


KIẾN TRÚC MICRO-FRONTEND

Phân tích demo cửa hàng trái cây
Theo mô hình 4+1 góc nhìn

Nguồn: micro-frontend-demo; mẫu Slidesgo

Micro-frontend chia giao diện theo nghiệp vụ

Chia theo nghiệp vụ

Danh mục, giỏ hàng, tài khoản là các trách nhiệm riêng.

Sở hữu tính năng

Nhóm quản lý tính năng và hợp đồng tích hợp.

Ghép trải nghiệm

Ứng dụng chủ kết hợp các phần thành một giao diện.

Tách thư mục hoặc giảm kích thước bundle chưa đủ để tạo ranh giới kiến trúc.

Nguồn: micro-frontends.org

Tách phát hành đòi hỏi hợp đồng tương thích

Một đơn vị phát hành

Một bản dựng

Danh mục + giỏ + tài khoản

Trình duyệt

Một giao diện thống nhất

Nhiều đơn vị phát hành

Danh mục

Giỏ

Tài khoản

Ứng dụng chủ + trình duyệt

Cần giữ hợp đồng tương thích

Nguồn: So sánh kiến trúc; micro-frontends.org

Cách tích hợp quyết định thời điểm ghép mô-đun

Thời điểm	Cơ chế	Điểm cần phối hợp
Khi dựng	Gói thư viện	Dựng lại bên sử dụng
Máy chủ / edge	Các mảnh HTML	Luồng ghép trên máy chủ
Lúc thực thi	Mô-đun từ xa	Hợp đồng khi chạy
Tài liệu riêng	Iframe	Trải nghiệm liên tài liệu

Demo sử dụng tích hợp lúc thực thi: Webpack 5 Module Federation.

Nguồn: Tổng hợp; tài liệu Webpack Module Federation

Tự chủ phát hành đi kèm chi phí tích hợp

Tự chủ

Lợi ích
Đổi từng tính năng.

Chi phí
Duy trì hợp đồng.

Tích hợp

Lợi ích
Tải phần cần dùng.

Chi phí
Xử lý chậm và lỗi.

Nhất quán

Lợi ích
Dùng chung trải nghiệm.

Chi phí
Phối hợp thư viện.

Nguồn: Phân tích kiến trúc định tính

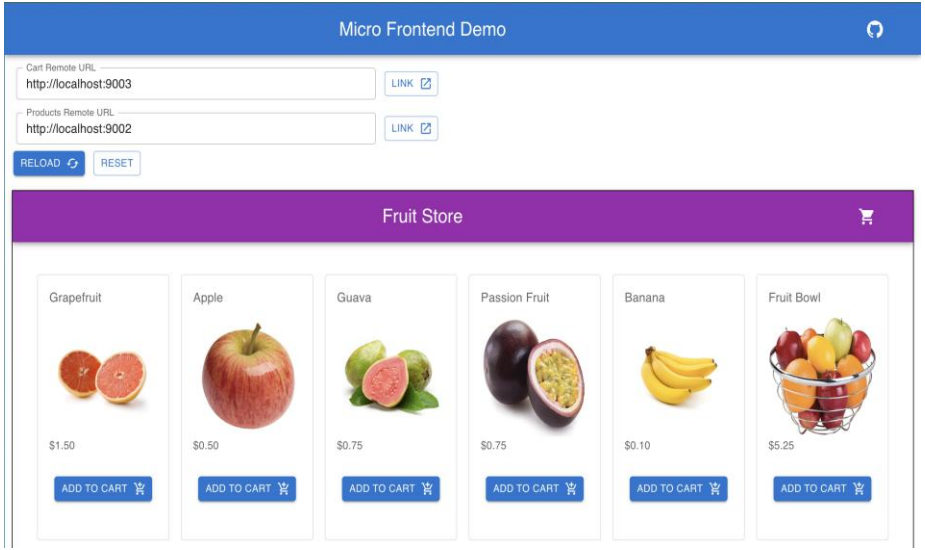
02

Phân tích demo thực tế

Ba bản dựng; một trải nghiệm
Kiểm tra ranh giới qua mã nguồn

Nguồn: Mã nguồn micro-frontend-demo

Cửa hàng ghép danh mục, giỏ hàng và thanh toán



MAIN — ứng dụng chủ

Khung trang, tuyến và URL

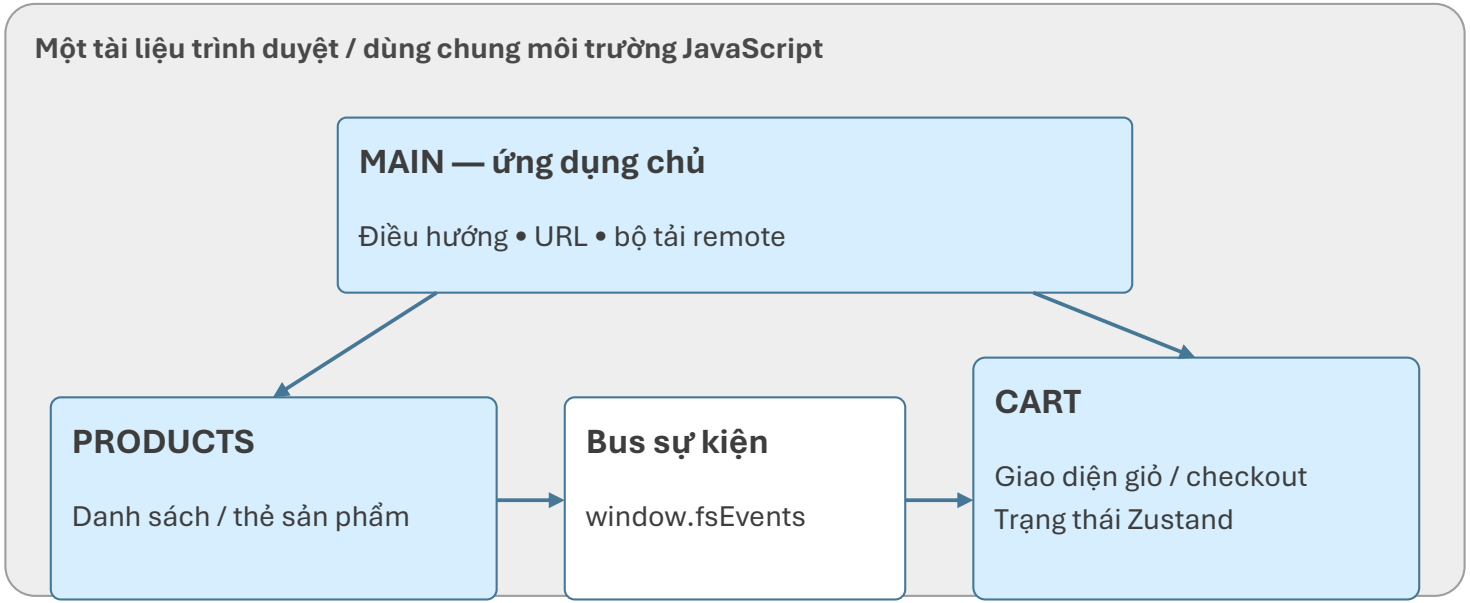
PRODUCTS — danh mục

Danh sách và thẻ sản phẩm

CART — giỏ và checkout

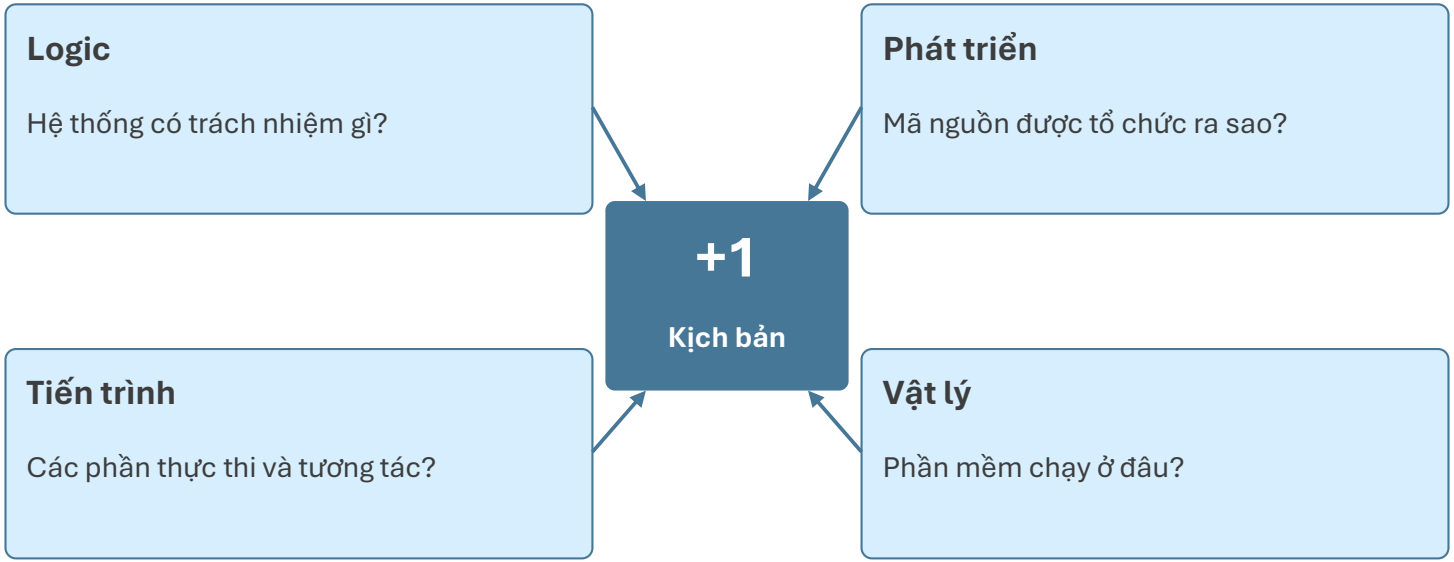
Nguồn: docs/demo.png; main/src/App.tsx

Ứng dụng chủ ghép hai remote trong trình duyệt



Nguồn: main/App.tsx; StoreHeader; RemoteComponent; cart/store

Mô hình 4+1 tách năm câu hỏi kiến trúc



Nguồn: Philippe Kruchten, mô hình 4+1, IEEE Software, 1995

Góc nhìn logic phân chia trách nhiệm chức năng

Ứng dụng chủ

Ghép giao diện
Quản lý tuyến
Tra cứu remote
Cung cấp bus

Danh mục

Hiển thị sản phẩm
Dựng thẻ sản phẩm
Phát lệnh thêm / xóa
Nhận ảnh chụp giỏ

Giỏ hàng

Quản lý số lượng
Lưu giỏ cục bộ
Dựng huy hiệu / ngăn
Tính tổng checkout

Phân chia chức năng rõ chưa đồng nghĩa với độc lập mã nguồn hoặc phát hành.

Nguồn: main/App.tsx; products/components; cart/components và store

Giao diện remote công khai gồm năm mô-đun

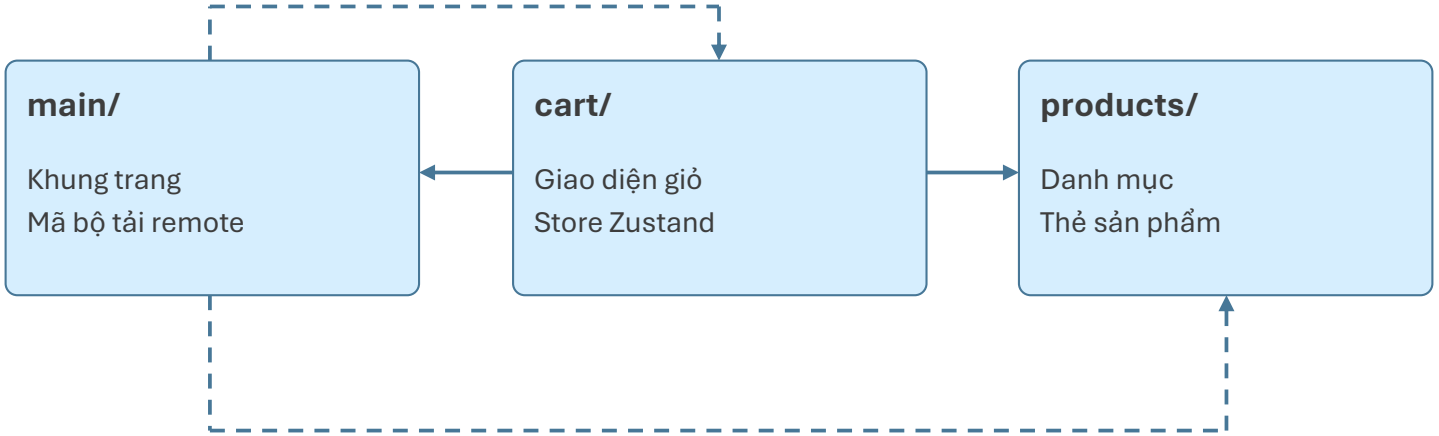
Container	Mô-đun công khai	Nơi sử dụng
PRODUCTS	./ProductsList	MAIN: tuyến /
PRODUCTS	./ProductCard	CART: ngăn giỏ, checkout
PRODUCTS	./products	Cart import thẳng nguồn
CART	./Cart	MAIN: StoreHeader
CART	./CheckoutPage	MAIN: /checkout

Hợp đồng gồm tên mô-đun, props và các thư viện dùng chung tương thích.

Nguồn: products/webpack.config.js; cart/webpack.config.js

Góc nhìn phát triển bộc lộ phụ thuộc mã nguồn

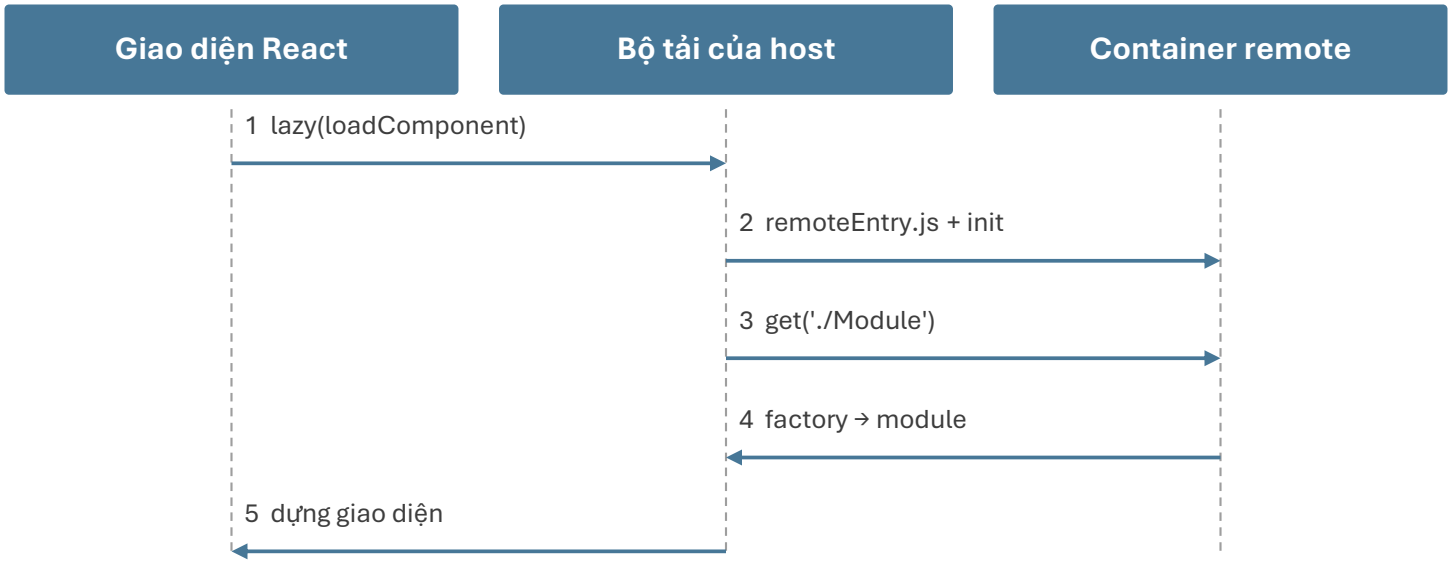
Một kho mã / Yarn workspaces / tác vụ Turborepo



Nét liền: import mã nguồn | Nét đứt: lấy mô-đun lúc chạy

Nguồn: package.json; cart/features/checkout; cart/components/CheckoutPanel

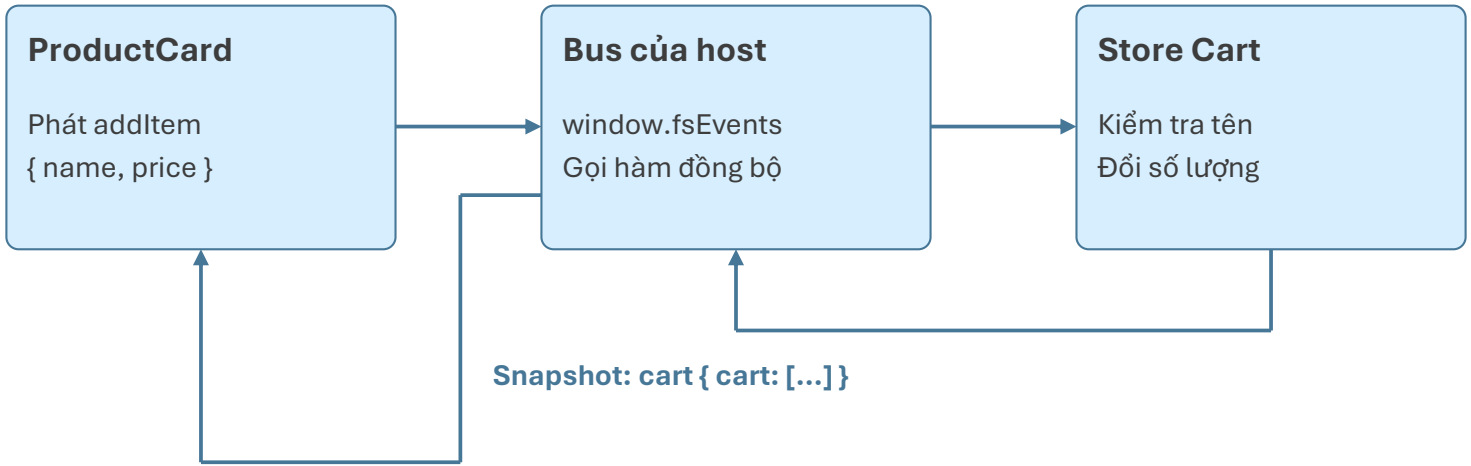
Góc nhìn tiến trình mô tả tải mã bất đồng bộ



Thời gian ↓ | Tải qua mạng; thực thi trong cùng tài liệu trình duyệt.

Nguồn: main/index.tsx; RemoteComponent; main/utils/index.ts

Lệnh qua bus; Cart sở hữu thay đổi trạng thái



Cập nhật thẻ sản phẩm

Lưu: fruit-shop-cart

Nguồn: ProductCard; main/services/pubsub.ts; cart/store/index.ts

Ba loại trạng thái có cơ chế lưu khác nhau

Cấu hình remote

Chủ sở hữu: MAIN

window.fsRemotes
localStorage:
fruit-remotes

Dữ liệu giỏ hàng

Chủ sở hữu: CART

Store Zustand
localStorage:
fruit-shop-cart

Thông điệp gần nhất

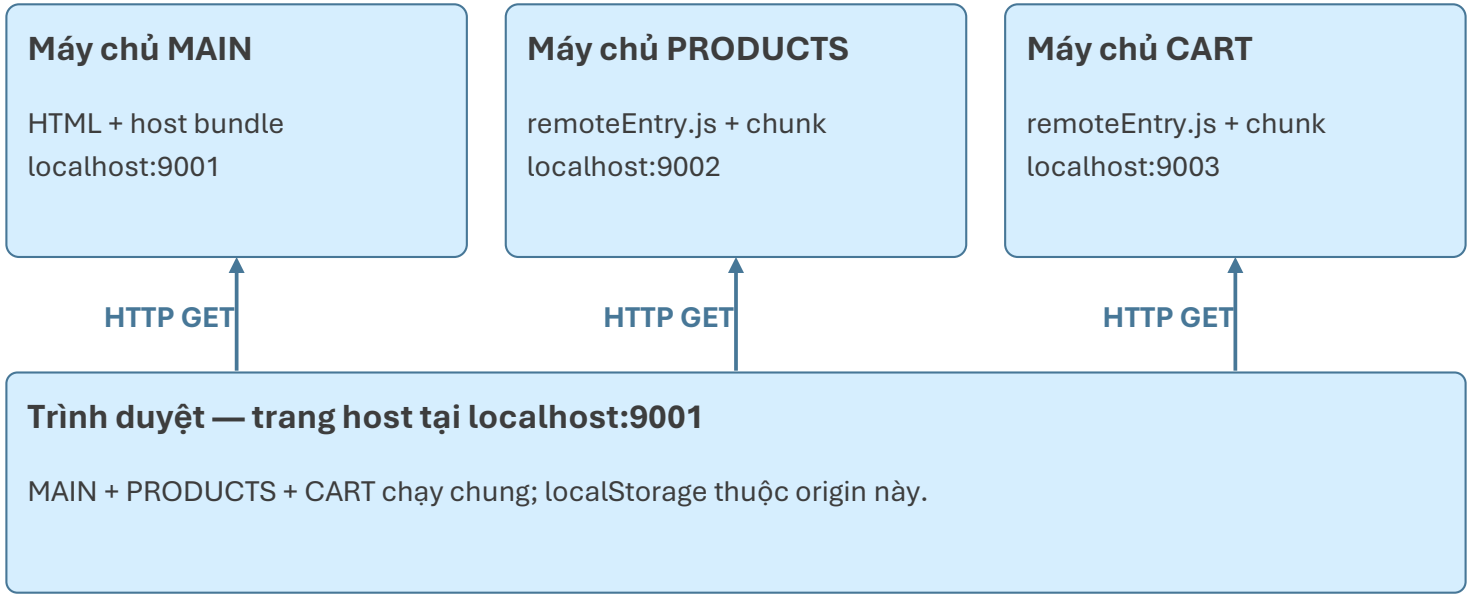
Chủ sở hữu: bus

window.fsEvents
Chỉ giữ trong bộ nhớ
Tạo lại sau reload

Origin của trang host quyết định phạm vi localStorage, dù mã đến từ remote khác.

Nguồn: main/context/remotes.tsx; PubSub; cart/store/index.ts

Ba máy chủ phân phối mã cho một runtime phía khách



Nguồn: webpack.config.js và lệnh start của ba ứng dụng

Đổi URL remote có hiệu lực sau khi tải lại trang

1 / Sửa URL

Chọn nơi phục vụ Products hoặc Cart.

2 / Lưu và tải lại

Ghi fruit-remotes.
Tải lại cả trang.

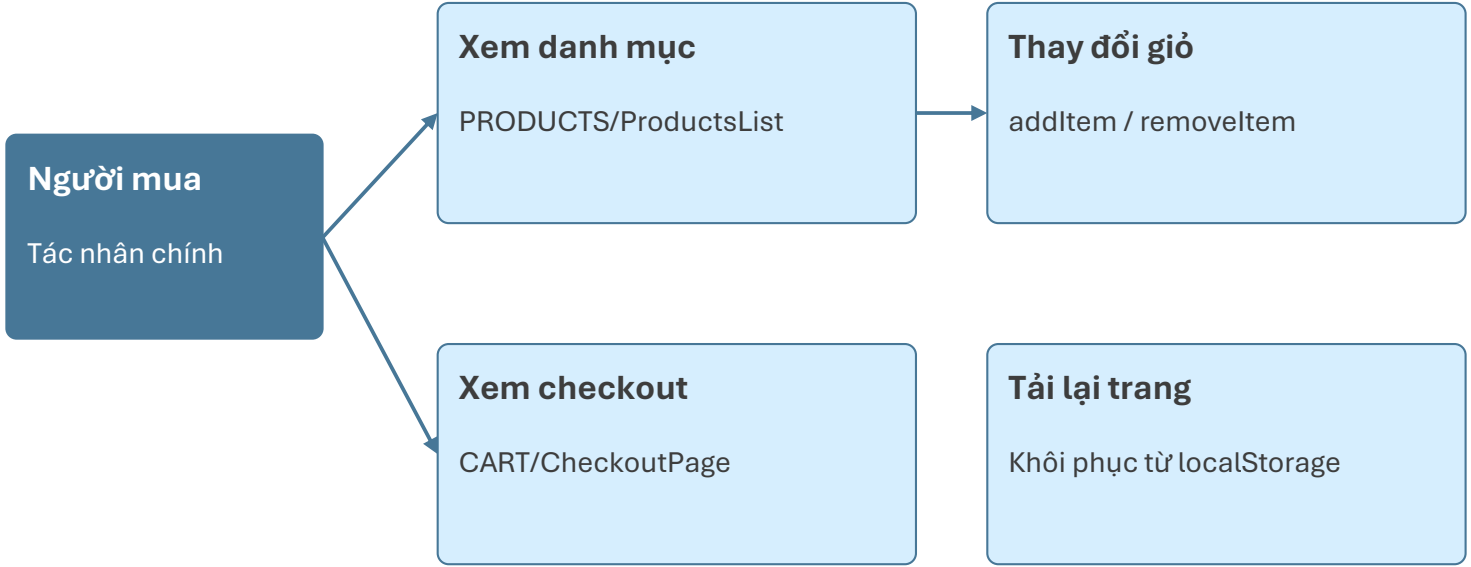
3 / Phân giải lại

Đọc registry.
Tải remoteEntry.js.

Chọn mã lúc chạy không thay thế được dữ liệu đã gắn vào bản dựng Cart.

Nguồn: RemoteControls; context/remotes.tsx; utils/index.ts

Kịch bản liên kết hành vi với bốn góc nhìn



Nhà phát triển: đổi URL → tải lại → lặp lại luồng mua hàng.

Nguồn: App.tsx; ProductCard; CheckoutPanel; RemoteControls

Luồng checkout đi qua đủ bốn góc nhìn

Góc nhìn	Điểm được quan sát
Logic	Chọn sản phẩm → giỏ → checkout
Phát triển	Cart phụ thuộc helper và bảng giá
Tiến trình	Lệnh → store → tuyến → tải thành phần
Vật lý	Trình duyệt lấy các tệp qua HTTP

Kết quả: mặt hàng và tổng tiền xuất hiện tại /checkout; chưa có thanh toán thật.

Nguồn: StoreHeader; CheckoutPanel; cart/features/checkout

Fallback giới hạn lỗi nhưng không cô lập mã

Remote chậm

Cơ chế
React Suspense

Giới hạn
Chưa có timeout rõ.

Tải / render lỗi

Cơ chế
ErrorBoundary cục bộ

Giới hạn
Chưa có điều khiển thử lại.

Remote gây sự cố

Bối cảnh
Chạy chung tài liệu

Giới hạn
Không có sandbox.

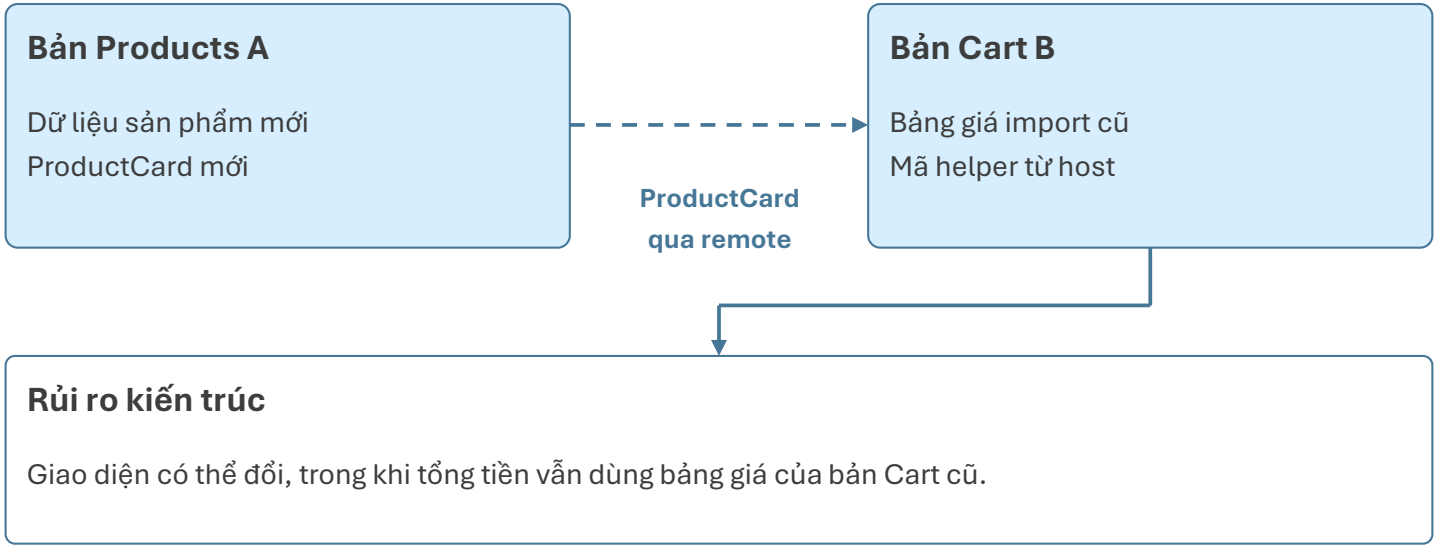
Nguồn: RemoteComponent; ErrorBoundary; utils/index.ts

Các quyết định cân bằng linh hoạt và ràng buộc

Quyết định	Lợi ích	Ràng buộc
Container động	Chọn nơi tải mã	Sẵn sàng và API
React singleton	Dùng chung runtime	Tương thích phiên bản
Bus của host	Tách bên phát / nhận	Schema và vòng đời
Zustand trong Cart	Rõ chủ sở hữu dữ liệu	Khôi phục và đồng bộ

Nguồn: Webpack configs; PubSub; remotes context; cart store

Thay remote không loại bỏ phụ thuộc lúc dựng



Nguồn: cart/features/checkout; cart/components/CheckoutPanel

Ba rủi ro trong mã làm yếu tính độc lập

Lệch cấu hình

Cart mặc định: 9002.

Lệnh start sửa: 9003.

Cấu hình rõ khi dựng.

Replay thiếu dữ liệu

Giá trị ban đầu: {}.

Bên nhận cần { cart }.

Khởi tạo đúng schema.

Đua lúc khởi động

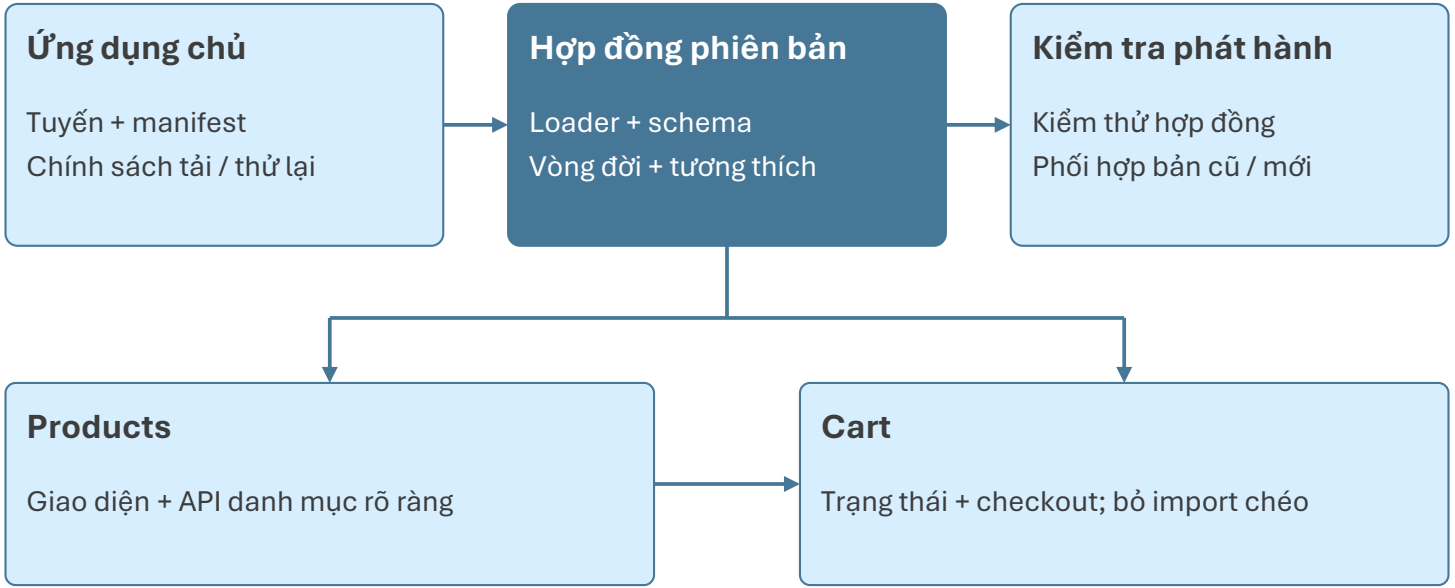
Chưa gộp promise tải.

Lệnh không có hàng đợi.

Kiểm tra sẵn sàng.

Nguồn: main/webpack.config.js; PubSub; main/utils/index.ts

Hợp đồng rõ giúp củng cố ranh giới phát hành



Nguồn: Đề xuất từ kết quả rà soát mã nguồn

Hai bản source có cách phục vụ khác nhau

Nguồn chính trong workspace

Webpack: MAIN :9001
PRODUCTS :9002; CART :9003

Bản copy trong repo ôn thi

Docker build → một dist / image
Nginx :9001; /products/; /cart/

Một tài liệu trình duyệt

Host tải mã remote qua HTTP; mọi remote chạy chung JavaScript.

Tách build và tải lúc chạy có thật; phát hành độc lập cần evidence riêng.

Nguồn: Workspace micro-frontend-demo; bản copy source/ trong repo ôn thi

Luồng demo ngắn làm rõ các ranh giới kiến trúc

1	**Khởi động**	Cài phụ thuộc; chạy yarn start trong thư mục demo.
2	**Theo dõi**	Mở :9001 → thêm hàng → giỏ → checkout → tải lại.
3	**Thay bản mã**	Đổi URL sang remote tương thích; chọn Reload.
4	**Thử lỗi**	Dùng URL không truy cập được; quan sát vùng lỗi.

Nguồn: README; lệnh start; RemoteControls; cart store

Nguồn: Tổng hợp phân tích mã nguồn demo

TÁCH BẢN DỰNG LÀ BƯỚC ĐẦU

Hợp đồng tương thích quyết định mức độ tự chủ

Thảo luận: đổi giá trong Products có cần dựng lại Cart?

Tài liệu gốc hỗ trợ các khái niệm kiến trúc

Micro-frontends — Michael Geers

https://micro-frontends.org/

Webpack — Module Federation

https://webpack.js.org/concepts/module-federation/

Philippe Kruchten — Mô hình 4+1 (1995)

https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf

mpuig — Công cụ agent-slides

https://github.com/mpuig/agent-slides

Nguồn: Các tài liệu chính thức được liên kết bên dưới

Bản đồ mã nguồn giúp kiểm tra các nhận định

Nội dung kiểm tra	Tệp mã nguồn
Ghép giao diện, tuyến	main/App.tsx; StoreHeader
Tìm và tải remote	main/context/remotes.tsx; utils/index.ts
Lệnh và replay	main/services/pubsub.ts; ProductCard
Trạng thái và lưu giỏ	cart/src/store/index.ts
Phụ thuộc mã nguồn	cart/.../checkout; CheckoutPanel
Exports và cổng	{main,products,cart}/webpack.config.js

Phạm vi: phân tích mã tĩnh; chưa kiểm thử hành vi của ứng dụng.

Nguồn: Kho mã micro-frontend-demo trong workspace

CẢM ƠN!

Câu hỏi và thảo luận

Một thay đổi trong Products có thể phát hành độc lập với Cart đến mức nào?

GHI CÔNG: Mẫu trình chiếu do Slidesgo, biểu tượng: Flaticon, đồ họa và hình ảnh: Freepik; minh họa: Stories

Nguồn: Mẫu Technology Consulting by Slidesgo; mã nguồn demo